

RECUT: recovering the causal state of an LLM

A new thesis hypothesis - not another output checker

Prepared for Nazish Baliyan and discussion with Prof. Abdulrahman Mahmoud. Research decision recorded on 16 September 2026, before implementation. RECUT means recovery using clean layer/token cuts. It is a working name, not a claim that no other project uses this acronym.

Central question: when a numerical error is discovered after an LLM has already used its cached memory, can we restore the entire correct computation while redoing substantially less work than ordinary rollback-and-replay?

In simple language, an LLM keeps a working memory called the key/value (KV) cache. A damaged entry can affect later computations. Fixing that original entry does not necessarily fix the later memory, and it cannot undo text already sent to a user. This project studies the recovery contract, not merely whether a checker raises an alarm.

The proposed mechanism keeps a short, private window of not-yet-committed tokens and selected intermediate hidden states. Following a localized error, it starts recomputation from a layer boundary whose inputs are still trustworthy. If the corrected next token differs, it stops reusing the old trajectory and regenerates the remaining suffix through every layer.

Why this is a better research question

It combines a precise correctness target, a systems cost tradeoff and a falsifiable experiment. The deliverable is not a renamed guardrail: it is a recovery algorithm, its assumptions, counterexamples to incomplete repairs, an executable reference and a matched-cost evaluation. Its eventual importance depends on whether it saves enough work after paying for checkpoints, journals, detection and delayed output.

This is the most defensible candidate from the present review, not a guaranteed high-impact result. The literature is crowded. A bounded pilot must be allowed to reject the performance claim.

Supervisor fit

Mahmoud's official profile emphasizes reliable, high-performance AI systems and hardware/software errors. His publication list includes GoldenEye (DSN 2022), selective CNN protection (ISSRE 2021), application-level soft-error analysis and Minotaur (ASPLOS 2019). Those methods connect directly to fault models, application-visible consequences and selective protection. RECUT extends that line to persistent autoregressive state rather than claiming the professor already works on this exact mechanism. This fit is stronger than a generic AI-safety chatbot project. [Official profile](#) and [publication list](#).

2. What exists, and what is left to test

The supplied collection contains 100 PDFs. Separate ledgers record section-level screening of their actual extracted text, with deeper review of closest neighbors. This is not a claim that all 100 papers received exhaustive line-by-line verification. Current primary-source searches extend beyond that collection. Program websites are discovery resources, not proof of novelty.

Prior work	Already established	Remaining hypothesis, if any
HCache, EuroSys 2025; HybridServe; KVPR	Hidden activations can reconstruct KV; compute, storage and I/O can be balanced.	Which saved states remain valid after a delayed numerical error and after a token change? Activation caching itself is not new.
Shared-KV bit-flip study, 2026	Fault persistence and propagation in shared caches; checksum-style containment.	Recovery after downstream use, conditional on trustworthy localization, not first discovery of propagation.
vLLM KV-load failure RFC	Affected requests can roll back to the longest valid prefix.	Can a layer/token frontier reduce recovery work beyond this strong baseline?
Delayed silent-error checkpointing, PRDC 2013	Verification delays, clean checkpoints, finite retention and checkpoint periods.	A transformer-specific frontier and its measured benefit, not generic rollback or journaling.
Models Take Notes; rollback-consistency work	Descendant cache state can retain old information; logical rollback need not restore internal state.	Exact numerical and token-trajectory restoration under explicit execution conditions.

Candidate contribution, stated narrowly

1. A two-dimensional recovery contract over layer and token position. The trusted region depends on both the earliest affected layer and whether the corrected token history still agrees with the speculative history.
2. A conservative recovery algorithm: reuse only certified-clean layer inputs while histories agree, then fall back to full-layer suffix regeneration at the first token divergence. The pilot assumes a trusted fault location and checkpoint; it does not solve detection.
3. A memory/latency frontier comparing no journal, sparse layer cuts and all-layer activation journals, including costs that can erase the benefit. The novelty would be the supported combination and transformer-specific insight, not any one familiar primitive.

Explicit novelty verdict

Conditional opening, not novelty clearance. No accessed source has yet established this exact delayed-corruption, trajectory-aware protocol and matched-cost result. That absence is not proof of priority. A supervisor and further related-work review must test the claim, including adaptations of ordinary checkpointing and activation restoration. Do not use "first", "provably optimal", "production-safe" or "A*-ready" on present evidence.

3. Correctness and feasibility

The contract

Use a deterministic, fixed-weight causal decoder, a verified prefix of T tokens and an uncommitted window of W decode transitions. An accidental finite perturbation affects a known KV layer L after the prefix checkpoint. Recovery computation, checkpoints and the chosen journal cut are trusted. The pilot uses benign tensor perturbations, not security exploits or hardware fault induction.

For a saved layer input at cut $c \leq L$, the value is clean while the token input history agrees with the fault-free execution: lower layers cannot depend on higher layers within a decoder step. Replaying layers c onward repairs affected descendants. If the corrected decision first differs at transition j , future token inputs change; all subsequent transitions must run from layer zero. Reusing their old hidden journals is invalid.

Induction over transitions provides the conditional argument: start from a clean prefix, reconstruct the same layer outputs for the same input, then either retain the matching next token or regenerate from the changed input. This depends on identical numerical execution semantics. Batch-prefill and incremental decoding are not assumed bitwise interchangeable. This is a proof sketch to formalize and test, not a completed formal verification.

What the experiment must include

Full suffix replay is the principal valid baseline. Restoring only the damaged slot is a deliberately incomplete negative control. A provenance-aware all-layer journal is a strong upper-bound baseline, not a straw man; it can start exactly at L but pays more journal storage. Sparse cuts should show a useful frontier, not necessarily beat all-layer recovery time.

Measure exact recovered token history, complete KV equality, final logits and subsequent continuation, plus recovery time, fault-free overhead, checkpoint bytes, actual journal bytes and output-commit delay. Report non-finite outcomes separately. Random trials must not select the worst possible fault location by searching model outputs.

HPC sufficiency

The student cluster provides RTX 5000 Ada workstations with approximately 32 GB GPU memory. A single allocated GPU is sufficient for the proposed compact-model pilot and likely subsequent single-GPU 1-3B experiments, subject to measured memory. It is not evidence of multi-GPU serving scalability. Use Slurm allocations only; no training, benchmarking or package installations on the login node. Existing RAVEN files remain untouched.

Why this matters - and when it does not

The intended setting is delayed integrity detection in long-lived inference state. If inexpensive verification before every read prevents all propagation, or errors are so rare that continuous journal overhead dominates every useful objective, RECUT may not justify deployment. Likewise, already-published outputs cannot be made unseen by cache repair. A holdback window is an application tradeoff, not free safety.

4. Publication decision and sources

Thesis versus conference paper

A thesis can credibly investigate this conditional mechanism, establish where it works and characterize failure boundaries. A selective conference paper requires stronger novelty and evidence than a correct small-model prototype. Potential communities include dependable computing, computer architecture and ML systems; choose a venue after the contribution is known, not from an acronym or promised ranking.

DSN/ISSRE are topical reliability candidates; MLSys or ASPLOS would require broader system impact and stronger implementation/scaling evidence. These are scope suggestions, not verified current ranking claims or acceptance predictions. No prototype can guarantee acceptance at an A or A* venue.

Pre-implementation go/no-go

Proceed with a bounded diagnostic pilot because the conditional correctness problem is precise and implementable. Upgrade to a full thesis only if correctness survives independent tests and sparse journals offer a worthwhile recovery/memory tradeoff. Reject a speed claim if recovery-only gains disappear under fault-free overhead, verification, checkpointing or commit latency. Publish negative findings only with a sufficiently novel and general explanation, not by relabeling failure as success.

Primary references checked before implementation

1. [HCache: Fast State Restoration in LLM Serving](#), EuroSys 2025 per author record.
2. [HybridServe: Activation Checkpointing and Hybrid Caching](#), 2025 author record; activation reconstruction is prior art.
3. [KVPR: I/O-Aware KV Cache Partial Recomputation](#), revised 2025.
4. [On the Combination of Silent Error Detection and Checkpointing](#), delayed-error checkpointing background.
5. [Bit-Flip Vulnerability in Shared KV Cache](#), 2026 author record; high-level comparison only.
6. [vLLM graceful KV-load failure RFC](#), engineering proposal, not peer-reviewed evidence.
7. [Models Take Notes at Prefill](#), preprint; downstream cache dependence.
8. [Aborted but Not Forgotten](#), preprint; defensive rollback contract only, no exploit reproduction.
9. [Stage-Replay Divergence Follows the KV Cache](#), preprint; numerical replay controls.
10. [Runtime-Certified Bounded-Error Quantized Attention](#) and [WitCert](#), preprints that rule out a generic runtime-certificate novelty claim.
11. [SmoLLM2 model card](#) and [Qwen2.5-0.5B model card](#), proposed public compact-model testbeds, not evidence of large-model generality.